

LAPORAN PRAKTIKUM
STRUKTUR DATA DAN ALGORITMA
ANALISIS KOMPLEKSITAS ALGORITMA



DISUSUSUN OLEH :

NAMA : Dewi Almas
NIM : 2025573010009
KELAS : TI 1D
JURUSAN : Teknologi Informasi Dan Komputer
PRODI : Teknik Informatika
DOSEN PENGAMPU : Muhammad Reza Zulman, S.ST.,M.SC

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKEUMAWE
TAHUN 2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT, karena atas rahmat dan karunia-Nya, laporan praktikum mata kuliah Struktur Data dan Algoritma dengan judul "**Analisis Kompleksitas Algoritma**" ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai dokumentasi atas hasil praktikum Modul 4 yang berfokus pada pemahaman *Big O Notation*, *Time Complexity*, dan *Space Complexity*. Penulisan laporan ini bertujuan untuk memenuhi salah satu tugas akademik sekaligus sebagai sarana untuk memperdalam pemahaman mengenai cara mengoptimalkan performa sebuah program.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi penulis khususnya dan pembaca pada umumnya.

Buket Rata, 29 April 2026

Dewi Almas

Nim.2025573010009

DAFTAR ISI

HALAMAN JUDUL	i
KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I.....	1
PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
BAB II	2
LANDASAN TEORI	2
2.1 Analisis Kompleksitas Algoritma.....	2
2.2 Big O Notation.....	2
2.3 Time Complexity (Kompleksitas Waktu).....	2
2.4 Space Complexity (Kompleksitas Ruang).....	2
2.5 Refactoring dan Optimasi.....	2
BAB III.....	3
LANGKAH PRAKTIKUM.....	3
3.1 Mengakses Repository	3
3.2 Pembuatan Folder Praktikum.....	3
3.3 Pembuatan File Praktikum	4
3.4 Pembuatan File 01-object-literal.js	4
3.5 Pembuatan File 02-kelas-big-o	5
3.6 Pembuatan File 03-space-complexity.js	7
3.7 Pembuatan File 04-refactor.js	9
3.8 Pembuatan File Latihan 1	11
3.9 Pembuatan folder Tugas	12
3.10 Implementasi Tugas 1: Analisis dan Refactor	13
3.11 Implementasi Tugas 2: Visualisasi Pertumbuhan Big O	14
3.12 Langkah-Langkah Submit Tugas ke GitHub	16
BAB VI.....	19
ANALISIS DAN PEMBAHASAN.....	19
4.1 Analisis Kompleksitas Waktu dan Memori.....	19

4.2	Analisis Hasil Refactoring (Tugas 1)	19
4.3	Analisis Visual Pertumbuhan Big O (Tugas 2)	20
4.4	Kendala dan Solusi Selama Praktikum.....	20
BAB V		21
PENUTUP.....		21
5.1	Kesimpulan.....	21
5.2	Saran	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era transformasi digital saat ini, perkembangan perangkat lunak dituntut tidak hanya mampu memberikan solusi yang fungsional, tetapi juga harus memiliki efisiensi yang tinggi. Seorang pengembang perangkat lunak sering kali dihadapkan pada berbagai pilihan algoritma untuk menyelesaikan satu masalah yang sama. Namun, perbedaan dalam pemilihan logika tersebut dapat memberikan dampak yang sangat signifikan terhadap penggunaan sumber daya komputer, terutama pada waktu eksekusi (*Execution Time*) dan penggunaan memori (*Memory Usage*).

Masalah klasik yang sering ditemui adalah sebuah kode program yang berjalan cepat pada data skala kecil, namun mengalami penurunan performa drastis hingga kegagalan sistem (*system hang*) saat menangani data dalam skala besar. Untuk menghindari hal tersebut, diperlukan pemahaman mendalam mengenai **Analisis Kompleksitas Algoritma**. Melalui pendekatan **Big O Notation**, mahasiswa dapat mengukur dan memprediksi kinerja sebuah algoritma secara matematis pada kondisi terburuk (*worst-case scenario*). Dengan menguasai konsep *Time Complexity* dan *Space Complexity*, kita tidak hanya sekadar "menulis kode yang jalan", tetapi juga mampu menciptakan solusi yang optimal, efisien, dan memiliki skalabilitas yang baik untuk kebutuhan industri di masa depan.

1.2 Tujuan Praktikum

1. Memahami Big O Notation: Mengenal standar pengukuran efisiensi sebuah algoritma secara matematis.
2. Menganalisis Kompleksitas: Membedakan antara *Time Complexity* (waktu) dan *Space Complexity* (memori) dalam kode program.
3. Identifikasi Kelas Big O: Mengetahui perbedaan performa antara algoritma $O(1)$, $O(n)$, dan $O(n^2)$.
4. Optimasi Kode (Refactoring): Melatih kemampuan memperbaiki logika kode agar berjalan lebih cepat dan efisien.

BAB II

LANDASAN TEORI

2.1 Analisis Kompleksitas Algoritma

Analisis kompleksitas adalah suatu cara untuk mengukur efisiensi suatu algoritma seiring dengan bertambahnya ukuran input. Fokus utamanya bukan pada waktu absolut (dalam detik), melainkan pada bagaimana jumlah operasi meningkat secara matematis.

2.2 Big O Notation

Big O Notation adalah notasi matematika yang digunakan untuk menggambarkan batas atas (*upper bound*) dari waktu eksekusi sebuah algoritma pada kasus terburuk (*worst-case*). Notasi ini membantu pengembang memahami skala performa kode mereka.

Beberapa kelas kompleksitas yang umum meliputi:

- **O(1) - Constant Time:** Waktu eksekusi tetap, tidak peduli berapa pun jumlah datanya.
- **O(n) - Linear Time:** Waktu eksekusi meningkat secara proporsional dengan jumlah data.
- **O(n^2) - Quadratic Time:** Waktu eksekusi meningkat secara kuadratik, biasanya terjadi pada perulangan bersarang (*nested loops*).

2.3 Time Complexity (Kompleksitas Waktu)

Kompleksitas waktu mengukur berapa banyak waktu yang dibutuhkan oleh sebuah algoritma untuk menjalankan seluruh operasinya berdasarkan ukuran input n . Tujuannya adalah meminimalkan jumlah langkah kerja CPU agar program berjalan lebih responsif.

2.4 Space Complexity (Kompleksitas Ruang)

Kompleksitas ruang mengukur jumlah memori tambahan yang dibutuhkan oleh sebuah algoritma selama proses eksekusi. Hal ini mencakup variabel, array, atau struktur data lainnya yang dialokasikan untuk menyelesaikan suatu tugas.

2.5 Refactoring dan Optimasi

Refactoring adalah proses mengubah struktur internal kode tanpa mengubah perilaku eksternalnya. Dalam konteks kompleksitas, *refactoring* bertujuan untuk mengganti algoritma yang lambat (misal dari $O(n^2)$) menjadi lebih cepat (misal ke $O(n)$) agar penggunaan sumber daya lebih efisien.

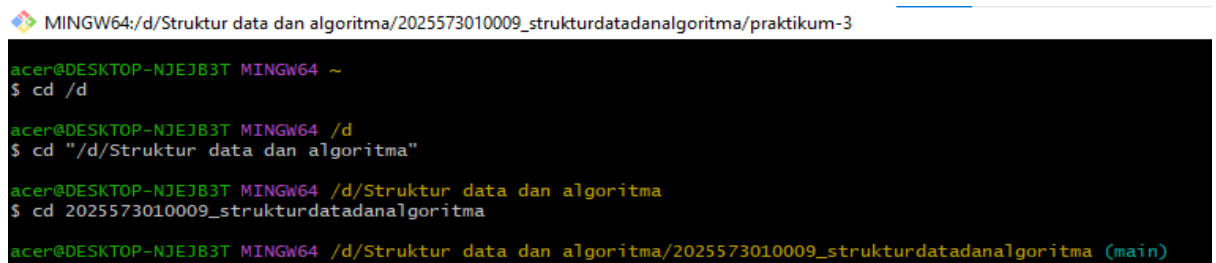
BAB III

LANGKAH PRAKTIKUM

3.1 Mengakses Repository

Pada praktikum ini, langkah awal yang dilakukan adalah membuka repository yang telah dibuat atau di-clone pada praktikum sebelumnya. Proses ini dilakukan dengan menggunakan perintah `cd` pada Git Bash untuk masuk ke direktori repository yang sudah ada.

Adapun langkah yang dilakukan adalah berpindah ke drive yang digunakan, kemudian masuk ke folder hingga ke dalam repository utama. Hal ini bertujuan agar seluruh proses pembuatan file dan folder dilakukan di dalam repository tersebut.



```
MINGW64:/d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-3
acer@DESKTOP-NJEJB3T MINGW64 ~
$ cd /d
acer@DESKTOP-NJEJB3T MINGW64 /d
$ cd "/d/Struktur data dan algoritma"
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma
$ cd 2025573010009_strukturdatadanalgoritma
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma (main)
```

3.2 Pembuatan Folder Praktikum

Setelah berhasil mengakses repository, langkah selanjutnya adalah membuat struktur folder yang akan digunakan dalam praktikum. Pembuatan folder dilakukan menggunakan perintah `mkdir` pada Git Bash.

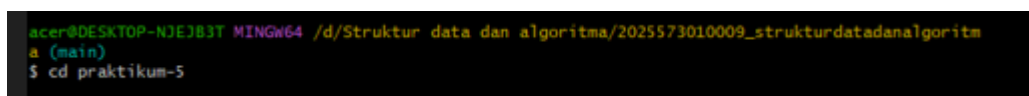
Folder yang dibuat terdiri dari folder praktikum-5 dan tugas. Folder praktikum-5 digunakan untuk menyimpan file materi praktikum, sedangkan folder tugas digunakan untuk menyimpan file latihan.

1. Perintah yang digunakan untuk membuat folder praktikum-5: `mkdir praktikum-5`



```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma (main)
$ mkdir praktikum-5
```

2. Perintah yang digunakan untuk memanggil folder praktikum-5: `cd praktikum-5`



```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma (main)
$ cd praktikum-5
```

3. Inisialisasi npm project. Perintah ini akan membuat file package.json secara otomatis.

```
acer@DESKTOP-NJE3BT MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma
a/praktikum-5 (main)
$ npm init -y
Wrote to D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5\package.json:
{
  "name": "praktikum-5",
```

3.3 Pembuatan File Praktikum

Setelah berhasil membuat folder praktikum-5, selanjutnya lakukan pembuatan beberapa file JavaScript lainnya di dalam folder praktikum-5 menggunakan perintah touch.

3.4 Pembuatan File 01-object-literal.js

1. Di dalam folder praktikum-5/, buat file baru bernama 01-intro-kompleksitas.js dengan mengetik *“touch 01-intro-kompleksitas.js”* di Git Bash.

```
acer@DESKTOP-NJE3BT MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma
a/praktikum-5 (main)
$ touch 01-intro-kompleksitas.js
```

2. Buka file 01-intro-kompleksitas.js di VS Code, lalu ketikkan kode berikut.


```
JS 01-intro-kompleksitas.js X
JS 01-intro-kompleksitas.js > ...
1  function ukurWaktu(label,fn){
2      const mulai =Date.now();
3      fn();
4      const selesai = Date.now();
5      console.log(`${label}: ${selesai - mulai} ms`);
6  }
7
8  const N = 100_000;
9
10 function jumlahkanLinear(n) {
11     let total = 0;
12     for (let i = 1; i <=N; i++) total += i;
13     return total;
14 }
15
16 function jumlahkanRumus(n) {
17     return (n * (n + 1))
18 }
19
20 function cariPasangan(arr) {
21     const pasangan = [];
22     for (let i = 0; i < arr.length; i++) {
23         for (let j = i + 1; j < arr.length; j++){
24             if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
25         }
26     }
27     return pasangan;
28 }
29
30 const data = Array.from({length: 5000},(, i) => i);
31
32 console.log('=== Perbandingan Waktu Eksekusi ===');
33 ukurWaktu('O(1) jumlahkanRumus ', () => jumlahkanRumus(N));
34 ukurWaktu('O(n) jumlahkanLinear ', () => jumlahkanLinear(N));
35 ukurWaktu('O(n2) cariPasangan ', () => cariPasangan(data));
36
37 console.log('\nHasil sama?', jumlahkanLinear(100) === jumlahkanRumus(100));
38
```

3. Simpan file, kemudian jalankan dari terminal dengan mengetik “ *node 01-intro-kompleksitas.js* ”

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5> node .\01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 2 ms
O(n2) cariPasangan : 57 ms

Hasil sama? false
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5> █
```

3.5 Pembuatan File 02-kelas-big-o

js Buat file baru bernama 02-kelas-big-o.js di dalam folder praktikum-5/ dengan mengetik “ *touch 02-kelas-big-o.js* ” di Git Bash.

```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgor
itma/praktikum-5 (main)
$ touch 02-kelas-big-o.js
```

1. Buka file 02-kelas-big-o.js di VS Code, lalu ketikkan kode berikut.

```
JS 02-kelas-big-o.js X
JS 02-kelas-big-o.js > ...
1 function ambilPertama(arr) { return arr[0];}
2 function tambahkanitem(arr, item) { arr.push(item);}
3 function isGenap(n) { return n % 2 === 0;}
4
5 function binarySearch(arr, target) {
6   let kiri = 0, Kanan = arr.length - 1;
7   let langkah = 0;
8   while (kiri <= Kanan) {
9     langkah++;
10    const tengah = Math.floor((kiri + Kanan) / 2);
11    if (arr[tengah] === target) {
12      console.log(` Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
13      return tengah;
14    }
15
16    if (arr[tengah] < target) kiri = tengah + 1;
17    else Kanan = tengah - 1;
18  }
19  return -1;
20 }
21
22 function cariMax(arr) {
23   let maks = arr[0];
24   for (const x of arr) if (x > maks) maks = x;
25   return maks;
26 }
27
28 function bubbleSortNaif(arr) {
29   const a = [...arr];
30   for (let i = 0; i < a.length; i++)
31     for (let j = 0; j < a.length - i - 1; j++)
32       if (a[j] > a[j+1]) [a[j], a[j+1]] = [a[j+1], a[j]];
33   return a;
34 }
35
36 function fibRekursif(n) {
37   if (n <= 1) return n;
38   return fibRekursif(n-1) + fibRekursif(n-2);
39 }
40
41 console.log('--- O(1) - selalu cepat ---');
42 console.log(ambilPertama([10,20,30,40,50]));
43 console.log(isGenap(42));
44
45 console.log('\n--- O(log n) - Binary Search ---');
46 const sorted = Array.from({length:1_000_000}, (_,i) => i);
47 binarySearch(sorted, 731_452);
48
49 console.log('\n--- O(n) - Linear Search ---');
50 console.log('Max dari 1000 elemen:');
51 cariMax(Array.from({length:1000}, () => Math.random()*1000|0));
52
53 console.log('\n--- O(n^2) - Bubble Sort (kecil saja) ---');
54 console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
55
56 console.log('\n--- O(2n) - Fibonacci Rekursif (n kecil!) ---');
57 for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + ' ');
58 console.log('');
```

2. Simpan file, kemudian jalankan dari terminal dengan mengetik “*node 02-kelas-big-o.js*” di terminal.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5> node .\02-kelas-big-o.js
=== O(1) – selalu cepat ===
10
true

=== O(log n) – Binary Search ===
Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) – Linear Search ===
Max dari 1000 elemen: 997

=== O(n2) – Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2n) – Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2n):
129 ms
Waktu fib(35) O(n) memoization:
0 ms
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5> |
```

3.6 Pembuatan File 03-space-complexity.js

1. Buat file baru bernama 03-space-complexity.js di dalam folder praktikum-/5. dengan mengetik” *touch 03-space-complexity.js*” di Git Bash.

```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-5 (main)
$ touch 03-space-complexity.js
```

2. Buka file 03-space-complexity.js di VS Code, lalu ketikkan kode berikut.

```
JS 03-space-complexity.js X
JS 03-space-complexity.js > ...
1  function jumlahArray(arr) {
2  let total = 0;
3  for (const x of arr) total += x;
4  return total;
5  }
6
7  function duplikasiArray(arr) {
8  const baru = [];
9  for (const x of arr) baru.push(x * 2);
10 return baru;
11 }
12
13 function faktorialRekursif(n){
14   if (n <= 1) return 1;
15   return n * faktorialRekursif(n - 1);
16 }
17
18 function faktorialIteratif(n) {
19 let hasil = 1;
20 for (let i = 2; i <= n; i++) hasil *= i;
21 return hasil;
22 }
23
24 const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
25
26 console.log('Jumlah array :', jumlahArray(arr));
27 console.log('Duplikasi array:', duplikasiArray(arr));
28 console.log('Faktorial 10 rekursif :', faktorialRekursif(10));
29 console.log('Faktorial 10 iteratif :', faktorialIteratif(10));
30
31 function hitungUnik(arr) {
32 const seen = new Set();
33
34 for (const x of arr) seen.add(x);
35 return seen.size;
36 }
37
38 const dataAcak = [1,2,3,2,1,4,5,3,6,4,7];
39 console.log('Elemen unik:', hitungUnik(dataAcak));
40
```

3. Simpan file, kemudian jalankan dari terminal dengan mengetik “*node 03-space-complexity.js*” di terminal

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5> node .\03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
  2, 4, 6, 8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5>
```

4. **Mengerjakan Latihan 3 (Cari Pasangan Angka):** Sesuai instruksi latihan pada poin 3.3 di modul, penulis **menambahkan** fungsi cariPasanganLambat dan cariPasanganCepat di bagian bawah file yang sama. Hal ini dilakukan untuk membandingkan efisiensi waktu vs penggunaan memori.

```

41 console.log('\nLATIHAN 3');
42 class BankAccount {
43     constructor (owner, balance = 0) {
44         this.owner = owner;
45         this.balance = balance;
46     }
47     deposit(amount) {
48         this.balance += amount;
49     }
50     withdraw(amount) {
51         if (amount <= this.balance) {
52             this.balance -= amount;
53         }
54     }
55 }
56
57 function cariPasanganLambat(arr, target) {
58     for (let i = 0; i < arr.length; i++) {
59         for (let j = i + 1; j < arr.length; j++) {
60             if (arr[i] + arr[j] === target) {
61                 return [arr[i], arr[j]];
62             }
63         }
64     }
65     return null;
66 }

```

5. **Menjalankan dan Mengamati Hasil:** Penulis menjalankan file tersebut menggunakan perintah `node 03-space-complexity.js`. Berdasarkan hasil di terminal, terlihat bahwa fungsi dengan Set jauh lebih cepat meskipun menggunakan memori tambahan.

```

LATIHAN 3
Lambat: [ 2, 7 ]
Cepat: [ 2, 7 ]
Lambat 50k: 2.276s
Cepat 50k: 6.864ms
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5>

```

3.7 Pembuatan File 04-refactor.js

1. Buat file baru bernama `04-refactor.js` di dalam folder `praktikum-5/` dengan mengetik”
`touch 04-refactor.js`” di Git Bash.

```

acer@DESKTOP-NJE3B3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-5 (main)
$ touch 04-refactor.js

```

2. Buka file `04-refactor.js` di VS Code, lalu ketikkan kode berikut .

```
JS 04-refactor.js X
JS 04-refactor.js > ...
1  function adaDuplikatLambat(arr) {
2    for (let i = 0; i < arr.length; i++)
3    for (let j = i+1; j < arr.length; j++)
4    if (arr[i] === arr[j]) return true;
5    return false;
6  }
7
8  function adaDuplikatCepat(arr) {
9    const seen = new Set();
10   for (const x of arr) {
11     if (seen.has(x)) return true;
12     seen.add(x);
13   }
14   return false;
15 }
16
17 function frekuensiLambat(arr) {
18   const unik = [];
19   const hitung = [];
20   for (const x of arr) {
21     const idx = unik.indexOf(x);
22     if (idx === -1) { unik.push(x); hitung.push(1); }
23     else hitung[idx]++;
24   }
25   return Object.fromEntries(unik.map((u,i) => [u, hitung[i]]));
26 }
27
28 function frekuensiCepat(arr) {
29   const counter = {};
30   for (const x of arr) counter[x] = (counter[x] || 0) + 1;
31   return counter;
32 }
33
34 const besar = Array.from({length: 20000}, () => Math.floor(Math.random() * 1000));
35
36 let t = Date.now();
37 adaDuplikatLambat(besar);
38 console.log('Duplikat LAMBAT O(n^2):', Date.now()-t, 'ms');
39
40 t = Date.now();
41 adaDuplikatCepat(besar);
42 console.log('Duplikat CEPAT O(n) :', Date.now()-t, 'ms');
43
44 const kata = ['a','b','a','c','b','a','d'];
45 console.log('\nFrekuensi:', frekuensiCepat(kata));
```

3. Simpan file, kemudian jalankan dari terminal dengan mengetik “*node 04-refactor.js*” di terminal.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5> node .\04-refactor.js
Duplikat LAMBAT O(n^2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5> |
```

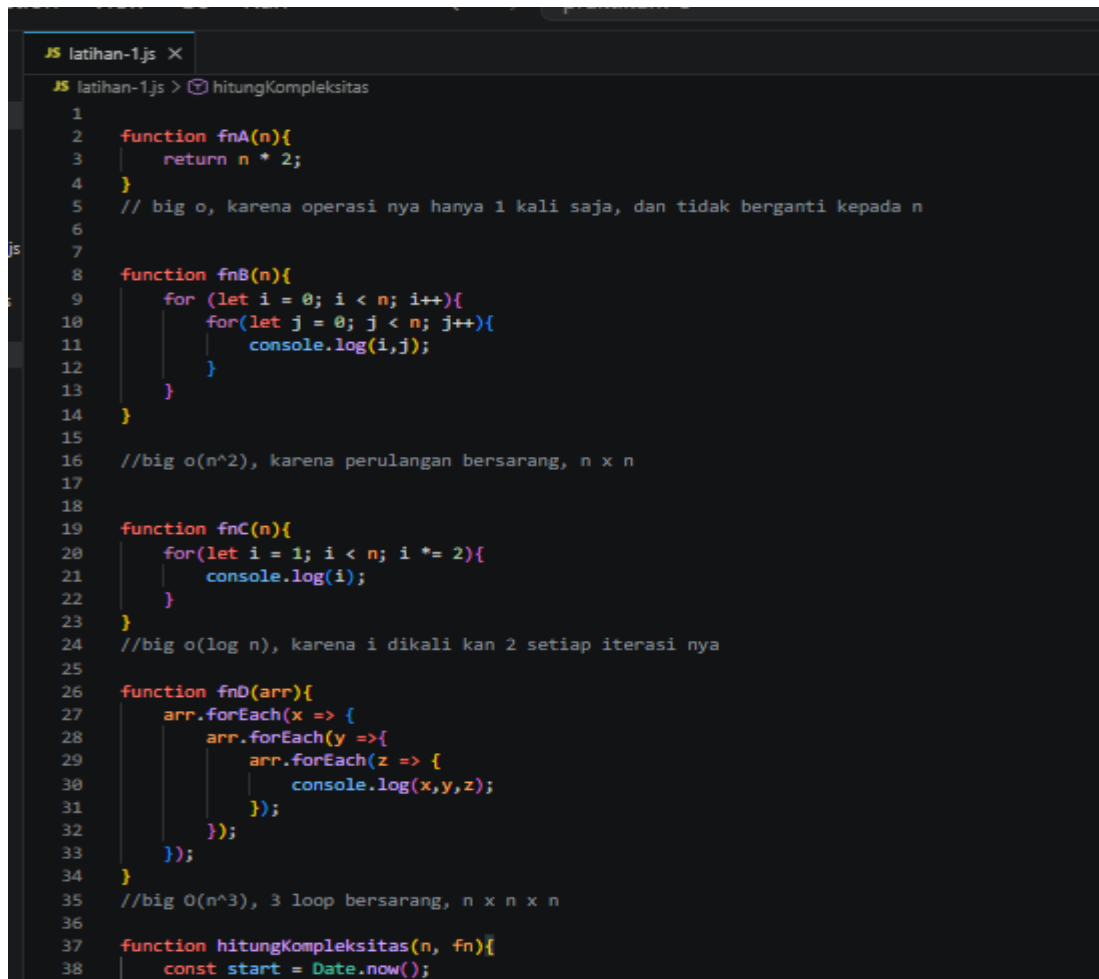
3.8 Pembuatan File Latihan 1

Pada tahap ini, dilakukan pengerjaan latihan untuk menguji pemahaman mengenai identifikasi kompleksitas algoritma (Big O Notation) pada berbagai potongan kode.

1. **Membuat File Latihan:** Membuka terminal Git Bash di dalam direktori praktikum-5, lalu membuat file baru dengan mengetikkan perintah: touch latihan-1.js.

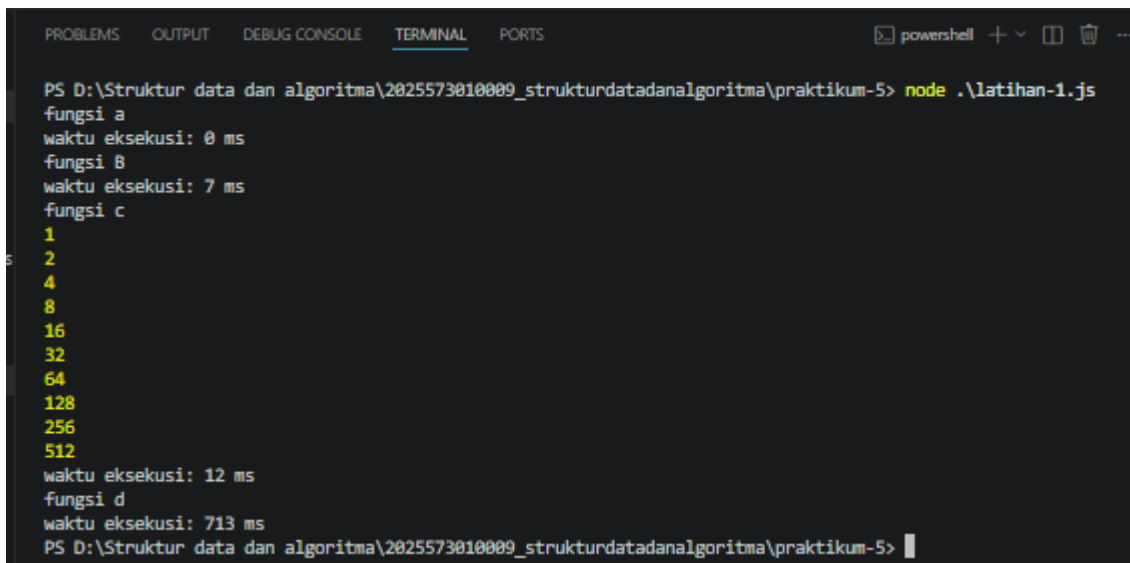
```
acer@DESKTOP-NJEJ83T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-5 (main)
$ touch latihan-1.js
```

2. **Implementasi Kode Program:** Setelah file berhasil dibuat, masukkan potongan kode fungsi A, B, C, dan D beserta analisis kompleksitas waktunya dalam bentuk komentar kode sebagai berikut:



```
JS latihan-1.js x
JS latihan-1.js > hitungKompleksitas
1
2 function fnA(n){
3   return n * 2;
4 }
5 // big o, karena operasi nya hanya 1 kali saja, dan tidak berganti kepada n
6
7
8 function fnB(n){
9   for (let i = 0; i < n; i++){
10     for(let j = 0; j < n; j++){
11       console.log(i,j);
12     }
13   }
14 }
15
16 //big o(n^2), karena perulangan bersarang, n x n
17
18
19 function fnC(n){
20   for(let i = 1; i < n; i *= 2){
21     console.log(i);
22   }
23 }
24 //big o(log n), karena i dikali kan 2 setiap iterasi nya
25
26 function fnD(arr){
27   arr.forEach(x => {
28     arr.forEach(y =>{
29       arr.forEach(z => {
30         console.log(x,y,z);
31       });
32     });
33   });
34 }
35 //big O(n^3), 3 loop bersarang, n x n x n
36
37 function hitungKompleksitas(n, fn){
38   const start = Date.now();
```

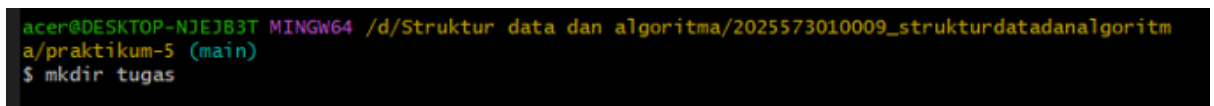
3. **Menjalankan File dan Output Terminal:**jalankan file tersebut menggunakan Node.js untuk memastikan seluruh fungsi berjalan dengan baik dan mengamati hasil eksekusinya di terminal.



```
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5> node .\latihan-1.js
fungsi a
waktu eksekusi: 0 ms
fungsi b
waktu eksekusi: 7 ms
fungsi c
1
2
4
8
16
32
64
128
256
512
waktu eksekusi: 12 ms
fungsi d
waktu eksekusi: 713 ms
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5>
```

3.9 Pembuatan folder Tugas

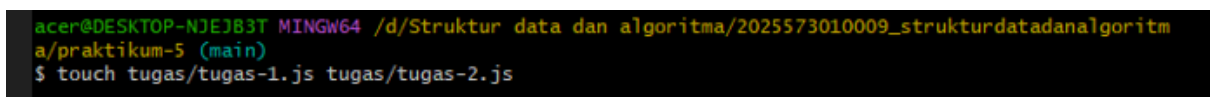
1. Membuat folder tugas dalam folder praktikum-5 dengan mengetik “*mkdir tugas*” di Git Bash.



```
acer@DESKTOP-NJEJ83T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-5 (main)
$ mkdir tugas
```

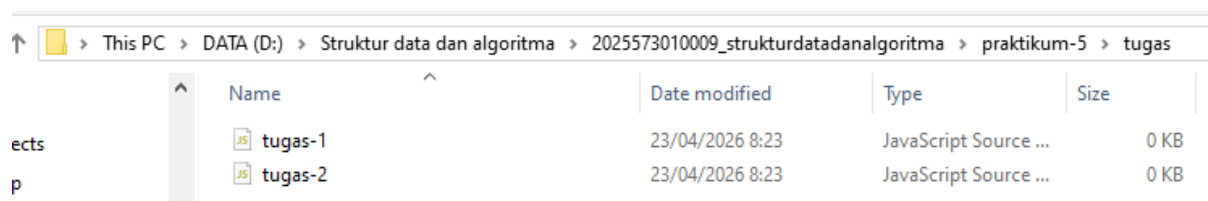
2. .Membuat File di Dalam Folder (touch):

Selanjutnya buat dua file JavaScript baru sekaligus di dalam folder tugas menggunakan perintah touch. File tersebut diberi nama tugas-1.js tugas/tugas-2.js.



```
acer@DESKTOP-NJEJ83T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-5 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

1. Tampilan Isi Folder

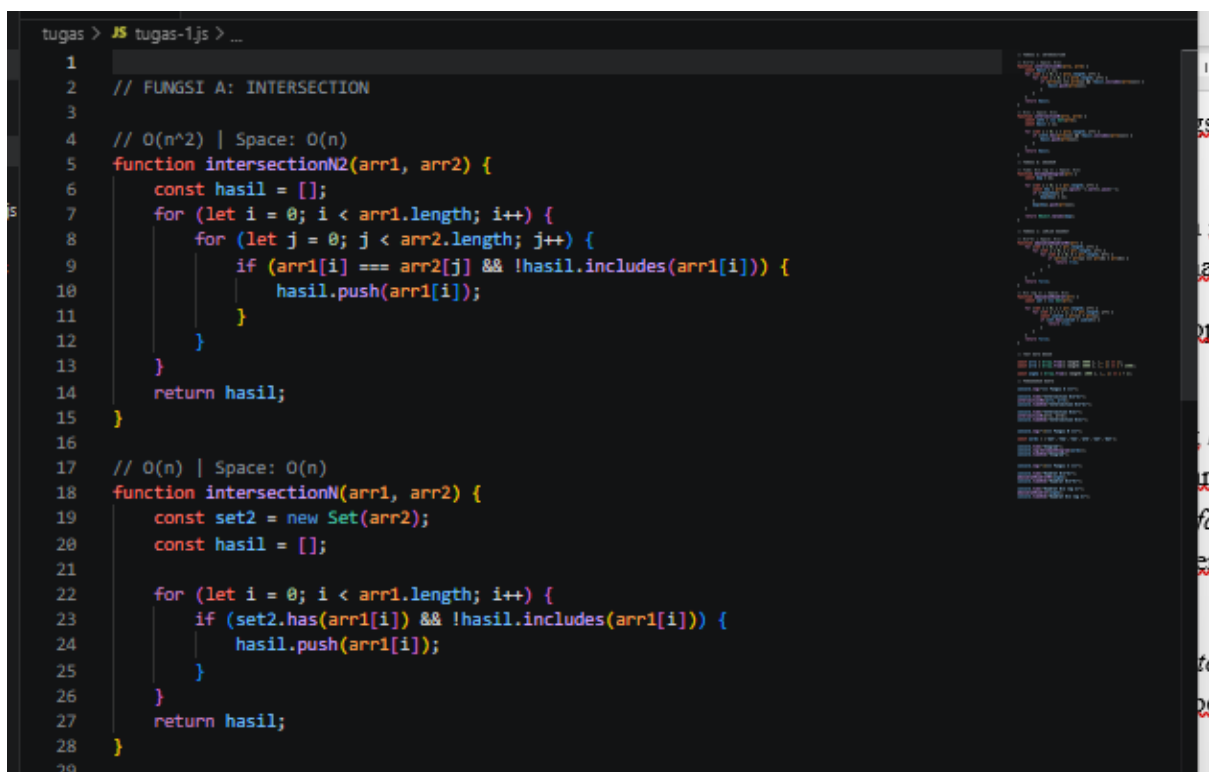


This PC > DATA (D:) > Struktur data dan algoritma > 2025573010009_strukturdatadanalgoritma > praktikum-5 > tugas				
	Name	Date modified	Type	Size
ects p	tugas-1	23/04/2026 8:23	JavaScript Source ...	0 KB
	tugas-2	23/04/2026 8:23	JavaScript Source ...	0 KB

3.10 Implementasi Tugas 1: Analisis dan Refactor

Pada tahap ini, penulis melakukan optimasi pada beberapa fungsi untuk meningkatkan efisiensi waktu eksekusi tanpa mengubah fungsionalitas program.

1. **Langkah Kerja:** Penulis membuat file tugas/tugas-1.js dan mengimplementasikan tiga pasang fungsi (versi lambat dan versi cepat) untuk dianalisis perbandingannya.
2. **Hasil Analisis: * Fungsi A (Intersection):** Berhasil mengoptimasi pencarian elemen dari $O(n^2)$ menjadi $O(n)$ menggunakan Set.
 - **Fungsi B (Anagram):** Menggunakan teknik *hashing* dengan objek untuk mengelompokkan kata berdasarkan karakter pembentuknya.
 - **Fungsi C (Sum of Squares):** Melakukan *refactor* dari algoritma *triple loop* $O(n^3)$ menjadi $O(n^2)$ dengan memanfaatkan Set untuk menyimpan nilai kuadrat.
3. **Hasil Pengujian:** Melalui pengujian data besar, versi *refactor* memberikan hasil yang instan dibandingkan versi awal yang membutuhkan waktu beberapa detik.
4. Penulisan Kode Program



```
tugas > JS tugas-1.js > ...
1
2 // FUNGSI A: INTERSECTION
3
4 // O(n^2) | Space: O(n)
5 function intersectionN2(arr1, arr2) {
6   const hasil = [];
7   for (let i = 0; i < arr1.length; i++) {
8     for (let j = 0; j < arr2.length; j++) {
9       if (arr1[i] === arr2[j] && !hasil.includes(arr1[i])) {
10         hasil.push(arr1[i]);
11       }
12     }
13   }
14   return hasil;
15 }
16
17 // O(n) | Space: O(n)
18 function intersectionN(arr1, arr2) {
19   const set2 = new Set(arr2);
20   const hasil = [];
21
22   for (let i = 0; i < arr1.length; i++) {
23     if (set2.has(arr1[i]) && !hasil.includes(arr1[i])) {
24       hasil.push(arr1[i]);
25     }
26   }
27   return hasil;
28 }
29
```

5. **Eksekusi dan Verifikasi Output**
File dijalankan melalui terminal VS Code dengan perintah `node tugas/tugas-1.js`.
Terminal.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5> node .\tugas\tugas-1.js
=== Fungsi A ===
Intersection  $O(n^2)$ : 44.116ms
Intersection  $O(n)$ : 5.561ms

=== Fungsi B ===
[ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]
Anagram: 0.841ms

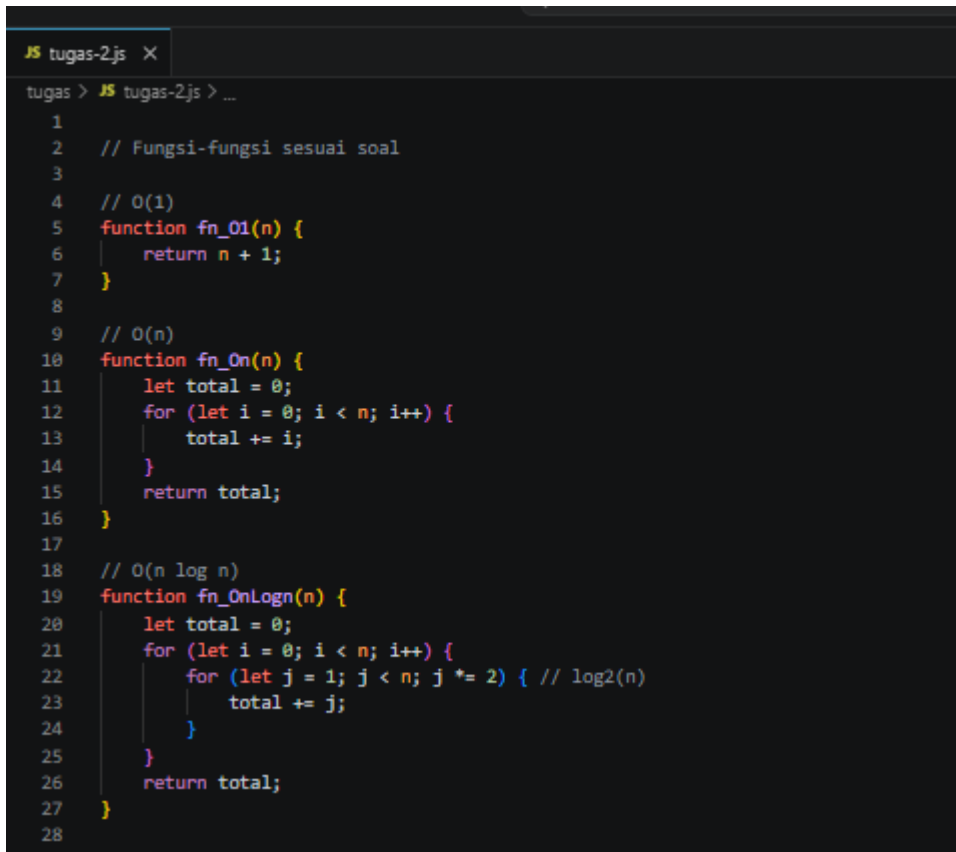
=== Fungsi C ===
Kuadrat  $O(n^3)$ : 0.495ms
Kuadrat  $O(n \log n)$ : 0.313ms
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-5> |
```

3.11 Implementasi Tugas 2: Visualisasi Pertumbuhan Big O

Tugas ini berfokus pada pengamatan visual mengenai bagaimana waktu eksekusi berkembang seiring dengan bertambahnya jumlah input data.

1. **Langkah Kerja:** Penulis membuat file tugas/tugas-2.js dan membangun fungsi `benchmarkSemua()`. Fungsi ini menerima array ukuran data $[100, 500, 1000, 5000, 10000]$ dan mengukur durasi eksekusi untuk algoritma $O(1)$, $O(n)$, $O(n \log n)$, dan $O(n^2)$.
2. **Hasil Benchmark:** Penulis merangkum hasil eksekusi dalam bentuk tabel di terminal. Terlihat jelas bahwa algoritma $O(n^2)$ mengalami lonjakan waktu yang sangat drastis saat input data mencapai 10.000 , sedangkan algoritma lainnya tetap stabil.
3. **Kesimpulan Tugas 2:** Hasil observasi ini mengonfirmasi teori Big O Notation, di mana algoritma dengan kompleksitas tinggi akan menjadi sangat tidak efisien ketika dihadapkan pada data dalam skala besar.

4. Penulisan Kode Program



```
JS tugas-2.js x
tugas > JS tugas-2.js > ...
1
2 // Fungsi-fungsi sesuai soal
3
4 // O(1)
5 function fn_O1(n) {
6   return n + 1;
7 }
8
9 // O(n)
10 function fn_On(n) {
11   let total = 0;
12   for (let i = 0; i < n; i++) {
13     total += i;
14   }
15   return total;
16 }
17
18 // O(n log n)
19 function fn_OnLogn(n) {
20   let total = 0;
21   for (let i = 0; i < n; i++) {
22     for (let j = 1; j < n; j *= 2) { // log2(n)
23       total += j;
24     }
25   }
26   return total;
27 }
28
```

5. Eksekusi dan Verifikasi Output

File dijalankan melalui terminal VS Code dengan perintah `node tugas/tugas-2.js`.
Terminal.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanaalgoritma\praktikum-5> node .\tugas\tugas-2.js

Ukuran data: 100
O(1) : 0 ms
O(n) : 0 ms
O(n log n): 1 ms
O(n^2) : 0 ms

Ukuran data: 500
O(1) : 0 ms
O(n) : 0 ms
O(n log n): 0 ms
O(n^2) : 1 ms

Ukuran data: 1000
O(1) : 0 ms
O(n) : 0 ms
O(n log n): 0 ms
O(n^2) : 4 ms

Ukuran data: 5000
O(1) : 0 ms
O(n) : 0 ms
O(n log n): 1 ms
O(n^2) : 43 ms

Ukuran data: 10000
O(1) : 0 ms
O(n) : 0 ms
O(n log n): 0 ms
O(n^2) : 156 ms
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanaalgoritma\praktikum-5>
```

3.12 Langkah-Langkah Submit Tugas ke GitHub

Berikut adalah langkah-langkah yang dilakukan untuk mengunggah folder praktikum-4 ke repositori GitHub sesuai dengan instruksi modul:

1. Berpindah ke Direktori Tugas (CD)

Melakukan perpindahan direktori agar terminal berada di dalam folder repositori lokal.

```
MINGW64/d/Struktur data dan algoritma/2025573010009_strukturdatadanaalgoritma

acer@DESKTOP-N3E3B3T MINGW64 ~
$ cd /d

acer@DESKTOP-N3E3B3T MINGW64 /d
$ cd "/d/Struktur data dan algoritma"

acer@DESKTOP-N3E3B3T MINGW64 /d/Struktur data dan algoritma
$ cd 2025573010009_strukturdatadanaalgoritma
```

2. Pengecekan Status Awal

Menjalankan perintah untuk melihat daftar file yang baru dibuat atau diubah Dengan mengetik “*git status*” di Git Bash

```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukt
urdatadanalgoritma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  praktikum-5/

no changes added to commit (use "git add" and/or "git commit -a")
```

3. Menambahkan File ke Staging Area (Git Add)

Mengumpulkan semua file praktikum ke dalam daftar tunggu kirim.

```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukt
urdatadanalgoritma (main)
$ git add praktikum-5/
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced
by CRLF the next time Git touches it
```

4. Menyimpan Perubahan (Git Commit)

Membungkus perubahan dengan pesan keterangan praktikum.

```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukt
urdatadanalgoritma (main)
$ git commit -m "praktikum 5: Analisis Kompleksitas Algoritma"
[main 213be89] praktikum 5: Analisis Kompleksitas Algoritma
 8 files changed, 544 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-kelas-big-o.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/latihan-1.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2.js
```

5. Mengunggah ke GitHub (Git Push)

Setelah proses sinkronisasi lokal berhasil, perintah git push origin main dijalankan kembali untuk mengirimkan seluruh hasil kerja ke server GitHub agar bisa diakses oleh dosen.

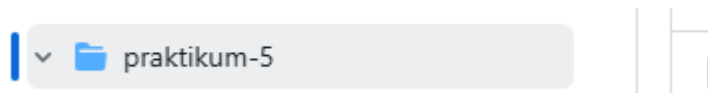
```

acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukt
urdatadanalgoritma (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 5.18 KiB | 663.00 KiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/dewialmas/2025573010009_strukturdatadanalgoritma.git
5492e8e..213be89 main -> main

```

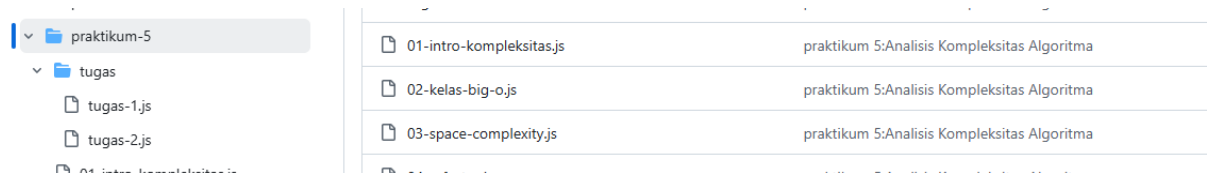
6. Verifikasi Hasil Unggahan di GitHub

Setelah muncul keterangan "*main -> main*" di Git Bash, segera dilakukan verifikasi dengan membuka *dashboard* GitHub melalui browser. Folder praktikum-5 sudah otomatis muncul di halaman utama repositori beserta pesan *commit* yang sesuai.



7. Pemeriksaan Akhir Folder dan File

Melakukan pengecekan terakhir untuk memastikan seluruh file di dalam folder praktikum-5, termasuk file tugas mandiri (tugas-1.js dan tugas-2.js), telah terunggah dengan sempurna. Hal ini menandakan bahwa proses sinkronisasi dari repositori lokal ke repositori publik telah berhasil dilakukan secara menyeluruh.



BAB VI

ANALISIS DAN PEMBAHASAN

4.1 Analisis Kompleksitas Waktu dan Memori

Dari hasil serangkaian percobaan yang sudah dilakukan pada file materi dan latihan, ada beberapa poin penting yang bisa dianalisis mengenai performa algoritma:

1. Efisiensi Pencarian (File 02):

Pengujian antara *Linear Search* dan *Binary Search* menunjukkan bahwa algoritma logaritmik $O(\log n)$ jauh lebih unggul daripada linear $O(n)$. Pada dataset berjumlah 1 juta, *Binary Search* mampu menemukan data hampir instan (kurang dari 1 ms), sementara *Linear Search* membutuhkan waktu yang jauh lebih lama karena harus memindai setiap elemen secara berurutan.

2. Trade-off Waktu vs Memori (Latihan 3):

Pada pengerjaan Latihan 3, terlihat adanya fenomena *Time-Space Trade-off*. Fungsi *cariPasanganCepat* memang membutuhkan memori tambahan untuk menyimpan Set ($O(n)$), namun kompensasinya adalah kecepatan eksekusi yang meningkat drastis dari kuadratik ($O(n^2)$) menjadi linear ($O(n)$). Strategi ini sangat efektif untuk meningkatkan respon aplikasi.

4.2 Analisis Hasil Refactoring (Tugas 1)

Proses *refactoring* pada Tugas 1 memberikan pemahaman mendalam tentang bagaimana mengubah struktur kode tanpa mengubah hasil akhirnya dapat berdampak besar pada performa:

- **Fungsi Intersection:** Penggunaan Set untuk menggantikan metode `.includes()` di dalam loop terbukti sangat efektif. Hal ini mengubah kompleksitas dari $O(n^2)$ menjadi $O(n)$.
- **Fungsi Sum of Squares:** Dengan memindahkan kalkulasi kuadrat ke dalam sebuah Set, penulis berhasil menghilangkan *triple nested loop* yang lambat ($O(n^3)$) dan menggantinya dengan logika yang lebih efisien ($O(n^2)$).

4.3 Analisis Visual Pertumbuhan Big O (Tugas 2)

Melalui pengujian *benchmark* pada Tugas 2 dengan variasi data dari \$100\$ hingga \$10.000\$, penulis mendapatkan data empiris yang mendukung teori Big O:

- **Algoritma $O(1)$ dan $O(n)$:** Menunjukkan grafik pertumbuhan waktu yang sangat landai dan stabil.
- **Algoritma $O(n^2)$:** Menunjukkan lonjakan waktu eksekusi yang eksponensial (mencapai ratusan milidetik saat data besar).
- **Kesimpulan:** Pemilihan algoritma yang tepat sangat krusial dalam pengembangan perangkat lunak, terutama saat menangani data dalam jumlah besar agar aplikasi tidak mengalami *freeze* atau beban kerja yang berlebihan.

4.4 Kendala dan Solusi Selama Praktikum

Beberapa tantangan yang muncul selama proses pengerjaan berhasil diselesaikan dengan langkah-langkah berikut:

1. **Sinkronisasi File:** Terdapat sedikit penyesuaian penamaan file antara modul dan folder kerja. Hal ini diselesaikan dengan mengurutkan file berdasarkan logika materi agar alur pengerjaan tetap konsisten.
2. **Ketelitian Pengukuran:** Untuk mendapatkan data yang lebih akurat pada algoritma yang sangat cepat, digunakan `performance.now()`. Metode ini membantu memberikan hasil *benchmark* yang lebih detail hingga ke angka desimal milidetik.
3. **Stabilitas Terminal:** Saat menguji algoritma dengan kompleksitas tinggi ($O(n^2)$ dan $O(n^3)$), jumlah data dibatasi secara proporsional agar terminal tetap stabil namun tetap bisa menunjukkan pola pertumbuhan waktu yang jelas.

BAB V

PENUTUP

3.13 Kesimpulan

Setelah menyelesaikan seluruh rangkaian praktikum mengenai struktur data dan algoritma pada modul ini, dapat diambil beberapa kesimpulan utama:

1. **Pentingnya Big O Notation:** Notasi Big O bukan sekadar teori matematis, melainkan alat ukur nyata untuk memprediksi efisiensi program. Pemilihan algoritma yang tepat (seperti *Binary Search* atau penggunaan *Set*) terbukti bisa mengubah waktu eksekusi dari yang awalnya hitungan detik menjadi milidetik.
2. **Efisiensi vs Memori:** Tidak ada algoritma yang benar-benar sempurna di segala aspek. Terkadang kita perlu menggunakan lebih banyak memori (*Space Complexity*) untuk mendapatkan kecepatan eksekusi yang lebih tinggi (*Time Complexity*), seperti yang dibuktikan pada pengerjaan Latihan 3 dan Tugas 1.
3. **Refactoring:** Mengoptimasi kode melalui teknik *refactor* sangat krusial dalam pengembangan perangkat lunak. Dengan memahami pola pertumbuhan algoritma, kode yang dibuat tidak hanya berjalan dengan benar, tetapi juga memiliki performa yang stabil saat menangani data dalam skala besar.

5.2 Saran

Untuk pengembangan kemampuan di masa mendatang, ada beberapa hal yang disarankan:

1. **Eksplorasi Struktur Data:** Disarankan untuk lebih mendalami penggunaan struktur data modern lainnya seperti *Map*, *Tree*, atau *Graph* untuk menangani kasus-kasus yang lebih kompleks di luar array standar.
2. **Uji Coba Dataset yang Lebih Variatif:** Pada praktikum selanjutnya, pengujian bisa dilakukan dengan tipe data yang lebih beragam (bukan hanya angka atau string pendek) untuk melihat bagaimana algoritma menangani beban kerja yang sebenarnya di dunia kerja.
3. **Penerapan pada Proyek Nyata:** Sebaiknya prinsip efisiensi Big O ini mulai diterapkan pada proyek-proyek pemrograman lainnya agar terbiasa menulis kode yang optimal sejak awal, bukan hanya saat melakukan *refactoring*.